Contents

1 Literature Review

1.1 AI-Based Intrusion Detection Systems for Next-Generation Networks

Comprehensive Review of Current State-of-the-Art
Last Updated: December 22, 2025

1.2 Table of Contents

1.3 1. Traditional Intrusion Detection Systems

1.3.1 1.1 Signature-Based IDS

Overview: Signature-based (or misuse-based) IDS rely on predefined patterns or signatures of known attacks. These systems match observed network traffic or system behavior against a database of attack signatures to identify malicious activity.

Key Systems: - **Snort [1]:** Open-source network intrusion detection system using rule-based pattern matching. Widely deployed in enterprise environments with extensive rule sets. - **Suricata [2]:** Multi-threaded successor to Snort with improved performance and protocol support. Supports IPS mode with inline deployment. - **YARA [3]:** Pattern matching tool primarily for malware detection but applicable to network traffic analysis.

Strengths: - High accuracy for known attacks (near 100% detection rate) - Low false positive rates (<1%) - Efficient processing with optimized pattern matching algorithms - Clear, interpretable alerts with specific attack identification - Well-established in industry with mature tooling

Limitations: - Cannot detect zero-day attacks or novel attack variants - Requires continuous signature updates (daily/weekly) - Vulnerable to polymorphic and metamorphic malware - Signature databases grow large, impacting performance - Evasion through obfuscation, encoding, fragmentation [4] - Reactive approach—detection only after attack is known

Research Gap: Need for proactive detection mechanisms that identify unknown threats without prior signatures.

1.3.2 1.2 Anomaly-Based IDS

Overview: Anomaly-based IDS establish a baseline of “normal” network behavior and flag deviations as potential intrusions. These systems can detect novel attacks but suffer from high false positive rates.

Approaches:

Statistical Methods [5]: - Threshold-based detection (mean, standard deviation) - Multivariate analysis (Mahalanobis distance, chi-square) - Time-series analysis (ARIMA, exponential smoothing) - *Limitation:* Assume normal distribution, struggle with complex patterns

Classical Machine Learning [6]: - k-Nearest Neighbors (k-NN): Instance-based learning - Support Vector Machines (SVM): Hyperplane separation with kernel tricks - Decision Trees: Rule-based classification with interpretable paths - Naive Bayes: Probabilistic classification assuming feature independence - *Achievement:* 85-95% accuracy on benchmark datasets (KDD'99, NSL-KDD)

Clustering Methods [7]: - K-means: Partition-based clustering - DBSCAN: Density-based spatial clustering - Hierarchical clustering: Agglomerative/divisive approaches - Self-Organizing Maps (SOM): Neural network-based clustering - *Use Case:* Unsupervised learning for discovering unknown attack patterns

Strengths: - Can detect zero-day attacks and novel intrusions - No signature database maintenance required - Adapts to network changes over time - Discovers previously unknown attack patterns

Limitations: - High false positive rates (10-30%) in practice [8] - Difficult to establish accurate baseline in dynamic environments - Sensitive to concept drift as network behavior evolves - Computational overhead for continuous monitoring - Requires extensive training data representing normal behavior - Difficulty distinguishing legitimate anomalies from attacks

Research Gap: Need for models that maintain low false positive rates while detecting unknown attacks.

1.3.3 1.3 Hybrid Approaches

Concept: Combine signature-based and anomaly-based detection to leverage strengths of both approaches.

Systems: - **Bro/Zeek [9]:** Hybrid network security monitor with policy-based scripting - **OSSEC [10]:** Host-based IDS with log analysis and anomaly detection - **Prelude [11]:** Hybrid IDS framework with correlation engine

Architecture:

```
Network Traffic → Signature Matching → Known Attack? → Alert
                        ↓ (No match)
                Anomaly Detection → Suspicious? → Alert (with confidence)
```

Evaluation [12]: - Detection rate: 92-96% (better than individual approaches) - False positive rate: 5-8% (moderate) - Processing overhead: 2-3× higher than signature-only

Limitations: - Inherits limitations from both paradigms - Increased computational complexity - Difficult to balance detection thresholds - Still struggles with adaptive, evolving threats

Research Gap: Need for intelligent fusion mechanisms that dynamically weight signature vs. anomaly detection.

1.4 2. AI/ML Techniques in Cybersecurity

1.4.1 2.1 Classical Machine Learning

1.4.1.1 2.1.1 Random Forest and Ensemble Methods Random Forest [13]:

Ensemble of decision trees with bootstrap aggregating (bagging) and random feature selection.

Application to IDS: - Ingale & Nasiruddin (2014): 99.67% accuracy on NSL-KDD with 100 trees [14] - Advantages: Handles high-dimensional data, resistant to overfitting, feature importance - Limitations: Computational cost for large datasets, black-box nature

Gradient Boosting [15]: - XGBoost: 98.5% accuracy on CICIDS2017 with optimized hyperparameters - LightGBM: Faster training with leaf-wise tree growth - CatBoost: Handles categorical features natively

AdaBoost [16]: Sequential ensemble focusing on misclassified instances.

Stacking [17]: Meta-learner combines predictions from multiple base models, achieving 2-3% accuracy improvement.

1.4.1.2 2.1.2 Support Vector Machines SVM Theory [18]:

Find optimal hyperplane maximizing margin between classes. Kernel trick enables non-linear separation.

Kernel Functions: - Linear: Fast but limited expressiveness - RBF (Gaussian): Most popular for IDS, handles non-linearity - Polynomial: Higher-degree relationships - Sigmoid: Neural network-like decision boundary

Performance [19]: - Binary classification: 95-98% accuracy on NSL-KDD - Multi-class: 90-93% accuracy (DoS, Probe, R2L, U2R) - Limitations: Slow training on large datasets ($O(n^3)$), sensitive to hyperparameters

One-Class SVM [20]: Unsupervised anomaly detection by learning boundary of normal class. - Precision: 85-90% but high false positives - Useful when attack samples are scarce

1.4.2 2.2 Deep Learning for IDS

1.4.2.1 2.2.1 Convolutional Neural Networks (CNN) CNN Architecture for Network Traffic [21]:

```
Traffic Matrix (Image-like) → Conv1D/2D → Pooling → Dense → Softmax
```

Key Research:

Kim et al. (2020) [22]: - 1D CNN on traffic flow features - Architecture: Conv1D(128) → MaxPool → Conv1D(256) → GlobalMaxPool → Dense(128) → Output - Results: 97.8% accuracy on CICIDS2017, 12ms inference time

Vinayakumar et al. (2019) [23]: - 2D CNN treating traffic as images - Data representation: Traffic flows as grayscale images (28×28, 64×64) - Results: 98.3% accuracy on NSL-KDD, better than classical ML

Advantages: - Automatic feature learning (no manual engineering) - Spatial pattern recognition in packet headers - Translation invariance (attack location in sequence) - Parallel processing on GPUs for speed

Limitations: - Requires large training datasets (100K+ samples) - Limited temporal modeling (need RNN for sequences) - Black-box nature (lack of interpretability) - Vulnerable to adversarial examples

1.4.2.2 2.2.2 Recurrent Neural Networks (RNN/LSTM/GRU) LSTM Architecture [24]:

```
Input → LSTM Cell (forget/input/output gates) → Hidden State → Output
```

Gate Mechanisms: - Forget gate: What to discard from cell state - Input gate: What new information to store - Output gate: What to output based on cell state

Key Research:

Yin et al. (2017) [25]: - LSTM for temporal modeling of network flows - Architecture: LSTM(128) → LSTM(64) → Dense(num_classes) - Results: 96.4% accuracy on NSL-KDD, captures temporal dependencies

GRU Variant [26]: - Simplified gating (reset + update gates) - Faster training than LSTM (fewer parameters) - Similar performance: 96.1% accuracy on CICIDS2017

Bidirectional LSTM [27]: - Process sequences forward and backward - Captures future context for better understanding - Results: 98.1% accuracy on UNSW-NB15

Advantages: - Models temporal dependencies in traffic sequences - Handles variable-length inputs naturally - Remembers long-term patterns (LSTM addresses vanishing gradient)

Limitations: - Sequential processing (slow compared to CNN) - Difficult to train (vanishing/exploding gradients) - Still black-box (limited interpretability)

1.4.2.3 2.2.3 Hybrid CNN-LSTM/GRU **Motivation:** Combine CNN's spatial feature extraction with LSTM's temporal modeling.

Architecture [28]:

```
Input Sequence → CNN (spatial features) → LSTM (temporal patterns) → Dense → Output
```

Key Research:

Lopez-Martin et al. (2020) [29]: - CNN-LSTM hybrid for network intrusion detection - CNN extracts features, LSTM models time dependencies - Results: 98.7% accuracy on CICIDS2017, F1-score: 98.2%

Xu et al. (2021) [30]: - CNN-GRU with attention mechanism - Attention focuses on important time steps - Results: 99.1% accuracy on NSL-KDD, 97.8% on UNSW-NB15

Advantages: - Best of both worlds: spatial + temporal - Outperforms standalone CNN or LSTM (2-4% improvement) - More robust to variations in attack patterns

Limitations: - Increased model complexity and training time - Higher computational requirements - Still lacks interpretability

1.4.2.4 2.2.4 Autoencoders for Anomaly Detection

Concept: Unsupervised learning to reconstruct normal traffic. High reconstruction error indicates anomaly.

Architecture [31]:

```
Encoder: Input → Latent Representation (bottleneck)
Decoder: Latent → Reconstructed Output
Loss: MSE(Input, Reconstructed)
```

Variants:

Stacked Autoencoder [32]: - Multiple encoding/decoding layers - Deep feature learning - Results: 94.6% detection rate on KDD'99

Variational Autoencoder (VAE) [33]: - Probabilistic latent space - Samples from learned distribution - Better generalization: 96.2% accuracy on UNSW-NB15

Adversarial Autoencoder [34]: - GAN-based regularization of latent space - More robust representations - Results: 97.1% detection rate with lower false positives

Advantages: - Unsupervised/semi-supervised (no labeled attacks needed) - Naturally detects outliers (novel attacks) - Compact representation in latent space

Limitations: - Threshold tuning for anomaly decision (trade-off: TPR vs. FPR) - Can struggle with complex attack patterns - Requires careful architecture design for different data types

1.4.3 2.3 Ensemble Methods in Deep Learning

Concept: Combine multiple deep learning models to improve robustness and accuracy.

Approaches:

Voting Ensemble [35]: - Hard voting: Majority class from multiple models - Soft voting: Weighted average of probabilities - Results: 98.8% accuracy combining CNN, LSTM, GRU

Stacking Ensemble [36]: - Base models: CNN, LSTM, Autoencoder - Meta-model: Random Forest or Neural Network - Results: 99.2% accuracy on CICIDS2017

Boosting for Deep Learning [37]: - AdaBoost with neural networks as weak learners - Gradient boosting with neural network base estimators

Advantages: - Improved accuracy and robustness (1-3% gain) - Reduced variance (model stability) - Better generalization to unseen attacks

Limitations: - Increased computational cost (N× models) - Higher memory requirements - Diminishing returns beyond 5-7 models

1.4.4 2.4 Reinforcement Learning for Adaptive IDS

Concept: Agent learns optimal defense policy through interaction with network environment.

Framework: - State: Network observations (traffic features, system state) - Action: Response decisions (block, allow, rate-limit, investigate) - Reward: Correct detection (+), false alarm (-), missed attack (- -) - Policy: Mapping from states to actions

Algorithms:

Q-Learning [38]: - Learn Q-value function: Q(s, a) = expected future reward - ε-greedy exploration strategy - Results: 94.3% detection rate with adaptive response

Deep Q-Network (DQN) [39]: - Neural network approximates Q-function - Experience replay for stable training - Target network for TD updates - Results: 96.7% detection, adapts to evolving threats

Actor-Critic Methods [40]: - Actor: Policy network (select actions) - Critic: Value network (evaluate actions) - A3C (Asynchronous Advantage Actor-Critic): Parallel training - Results: 97.2% detection with 15% faster adaptation

Proximal Policy Optimization (PPO) [41]: - More stable policy updates (clipped objective) - Better sample efficiency than A3C - Results: 97.8% detection, smoother learning curve

Advantages: - Adaptive to evolving threats (online learning) - Learns optimal response strategies - Balances exploration vs. exploitation - Can handle partial observability (POMDP)

Limitations: - Requires extensive training (millions of interactions) - Difficult reward engineering - Sample inefficiency in sparse reward scenarios - Safety concerns during exploration

---

1.5 3. Next-Generation Network Security

1.5.1 3.1 5G/6G Security

5G Architecture Vulnerabilities [42]:

Network Slicing: - Isolation failures between slices - Resource starvation attacks - Slice hijacking through control plane exploitation

Edge Computing: - Increased attack surface at edge nodes - Physical tampering risks - Distributed denial-of-service from compromised edges

Massive IoT: - Billions of low-security devices - Amplification attacks using IoT botnets - Signaling storms from misbehaving devices

Key Research:

Ahmad et al. (2019) [43]: - Comprehensive survey of 5G security threats - Identified 50+ unique vulnerabilities across layers - Proposed ML-based anomaly detection for

5G core

Li et al. (2020) [44]: - Deep learning IDS for 5G network slicing - CNN-LSTM hybrid for slice-specific attack detection - Results: 97.1% accuracy, <50ms detection latency

Ferrag et al. (2021) [45]: - Federated learning for distributed 5G IDS - Privacy-preserving collaborative threat intelligence - Results: 96.8% accuracy without centralizing data

Research Gaps: - Limited real 5G attack datasets (mostly simulated) - Scalability to millions of network slices - Real-time processing at 10+ Gbps speeds - Cross-slice attack detection

1.5.2 3.2 IoT Security

IoT Threat Landscape [46]:

Device Constraints: - Limited CPU (ARM Cortex-M, 50-200 MHz) - Restricted memory (32-512 KB RAM) - Power constraints (battery-operated) - Minimal security capabilities

Attack Types: - Mirai botnet (DDoS amplification) - Brute-force attacks (default credentials) - Firmware exploitation (buffer overflows) - Physical tampering - Side-channel attacks

Key Research:

Diro & Chilamkurti (2018) [47]: - Distributed deep learning IDS for IoT fog computing - Lightweight model on edge, full model in fog - Results: 98.27% accuracy on BoT-IoT dataset

HaddadPajouh et al. (2018) [48]: - LSTM-based IoT malware detection - Analyzes system call sequences - Results: 98.18% accuracy on IoT-23 dataset

Koroniotis et al. (2019) [49]: - Bot-IoT dataset with realistic IoT botnet traffic - Evaluated 9 ML algorithms - Best: Random Forest with 99.94% accuracy (may indicate overfitting)

Lightweight Solutions:

MobileNet for IoT IDS [50]: - Depthwise separable convolutions - 10× fewer parameters than standard CNN - Results: 96.3% accuracy with 5MB model size

Binary Neural Networks [51]: - 1-bit weights and activations - 32× memory reduction - Results: 94.1% accuracy (slight degradation for efficiency)

Research Gaps: - Energy-efficient IDS for battery-powered devices - Real-time detection on resource-constrained MCUs - Heterogeneous IoT protocol support (Zigbee, Z-Wave, LoRa) - Scalability to billions of devices

1.5.3 3.3 Software-Defined Networks (SDN)

SDN Architecture [52]:

```
Application Layer (Security Apps)
        ↕ Northbound API
Control Layer (SDN Controller: ONOS, ODL)
        ↕ Southbound API (OpenFlow)
Data Layer (OpenFlow Switches)
```

SDN Attack Vectors [53]:

Control Plane: - Controller DoS (overwhelming with requests) - Flow table overflow - Topology poisoning - Man-in-the-middle on control channel

Data Plane: - Switch compromise - Flow rule manipulation - Packet flooding

API Vulnerabilities: - Unauthorized access to northbound APIs - Southbound protocol exploits (OpenFlow)

Key Research:

Tang et al. (2020) [54]: - Deep RNN for SDN intrusion detection - Real-time analysis of OpenFlow statistics - Results: 95.74% accuracy, 23ms detection time

Elsayed et al. (2020) [55]: - InSDN dataset with realistic SDN attacks - Comprehensive feature set (104 features) - Evaluated 8 ML algorithms, best: Random Forest (98.92%)

Ashraf & Latif (2020) [56]: - Machine learning for SDN DDoS detection - Decision tree-based classifier at controller - Results: 99.8% accuracy with 10ms response time

Distributed SDN IDS [57]: - Multi-controller architecture - Federated learning across controllers - Load balancing and resilience - Results: 97.5% accuracy with horizontal scalability

Research Gaps: - Protection against controller-targeted attacks - Scalability to large-scale data center SDN - Multi-domain SDN security (inter-controller) - Integration with NFV security

1.5.4 3.4 Edge Computing Security

Edge Computing Paradigm [58]:

```
Cloud (Centralized processing, storage)
  ↕
Fog (Regional aggregation, coordination)
  ↕
Edge (Local processing, low latency)
  ↕
IoT Devices (Data generation)
```

Security Challenges:

Distributed Attack Surface: - Thousands of edge nodes, each a potential target - Physical security risks (public locations) - Heterogeneous devices and platforms

Resource Constraints: - Limited compute for complex IDS models - Bandwidth constraints for cloud communication - Power limitations

Data Privacy: - Sensitive data processed at edge - Multi-tenancy security issues - Compliance with regional data laws (GDPR)

Key Research:

Preuveneers et al. (2018) [59]: - Distributed IDS at fog layer - Anomaly detection with federated learning - Results: 94.6% accuracy without centralizing data

Chen et al. (2019) [60]: - Lightweight deep learning for edge IDS - Knowledge distillation from complex teacher - Results: 96.1% accuracy with 10× speedup

Xiao et al. (2023) [61]: - EdgeIDS framework for real-time IoT protection - Hybrid edge-cloud architecture - Results: 98.2% accuracy, <30ms latency

Research Gaps: - Model compression for extreme edge (MCU-level) - Hierarchical coordination (edge-fog-cloud) - Privacy-utility trade-offs in federated scenarios - Real-time updates to edge models

---

1.6 4. Recent Advances (2019-2025)

1.6.1 4.1 Transformer Networks for IDS

Background: Transformers, introduced in "Attention Is All You Need" [62], revolutionized NLP. Their self-attention mechanism captures long-range dependencies without sequential processing.

Application to IDS:

Architecture:

```
Traffic Sequence → Token Embedding → Positional Encoding
  → Multi-Head Self-Attention → Feed-Forward → Classification
```

Key Research:

Lin et al. (2022) [63]: - BERT-based IDS for SDN - Pre-training on normal traffic, fine-tuning for attack detection - Results: 98.9% accuracy on InSDN dataset

Zhang et al. (2021) [64]: - Transformer encoder for network traffic classification - Multi-head attention focuses on critical packet features - Results: 99.2% accuracy on CICIDS2018

Advantages: - Parallel processing (faster than RNN/LSTM) - Captures global dependencies (entire traffic sequence) - Attention weights provide interpretability - State-of-the-art results on sequence tasks

Limitations: - Computational complexity $O(n^2)$ for sequence length n - Requires large datasets for effective training - Memory intensive for long sequences - Position encoding may not suit all network patterns

Research Gap: Efficient transformers for high-speed networks (sparse attention, linear attention).

1.6.2 4.2 Federated Learning for Privacy-Preserving IDS

Concept [65]: Train global model collaboratively without centralizing data. Each node trains locally, shares only model updates.

Process:

```
1. Server distributes initial model to clients
2. Clients train on local data
3. Clients send model updates (gradients/weights) to server
4. Server aggregates updates (FedAvg, FedProx)
5. Server distributes updated global model
6. Repeat
```

Key Research:

Zhao et al. (2020) [66]: - Federated learning for IoT intrusion detection - Blockchain-based secure aggregation - Results: 96.5% accuracy with privacy preservation

Nguyen et al. (2022) [67]: - Comprehensive survey on federated IDS - Identified challenges: non-IID data, communication cost, adversarial clients - Solutions: personalized federated learning, compression, Byzantine-robust aggregation

Mothukuri et al. (2021) [68]: - Security and privacy of federated learning itself - Attacks: Model poisoning, inference attacks, backdoor injection - Defenses: Differential privacy, secure aggregation, client selection

Advantages: - Data privacy (complies with GDPR, HIPAA) - Scalable to distributed networks - Leverages diverse data from multiple sources - Reduces bandwidth (model updates < raw data)

Limitations: - Communication overhead (model updates every round) - Non-IID data distribution across clients - Vulnerability to poisoning attacks - Slow convergence compared to centralized

Research Gaps: - Efficient aggregation for large models - Handling extreme non-IID scenarios - Byzantine-robust algorithms for adversarial clients - Personalization for client-specific threats

1.6.3 4.3 Explainable AI (XAI) for IDS

Motivation: Black-box ML models lack trust in security contexts. Analysts need to understand why a decision was made.

Techniques:

SHAP (SHapley Additive exPlanations) [69]: - Game-theoretic approach to feature importance - Consistent, locally accurate explanations - Global feature importance via aggregation

LIME (Local Interpretable Model-agnostic Explanations) [70]: - Perturb input, observe output changes - Fit local linear model around instance - Fast, model-agnostic

Attention Mechanism [71]: - Visualize attention weights in neural networks - Identify which traffic features/timestamps influenced decision - Built into Transformer, CNN, LSTM with attention

Layer-wise Relevance Propagation (LRP) [72]: - Backpropagate prediction through network layers - Assign relevance scores to input features - Specific to neural architectures

Key Research:

Islam et al. (2021) [73]: - Survey of XAI approaches for cybersecurity - Evaluated SHAP, LIME, attention for IDS - Found SHAP most consistent, LIME fastest

Kuppa et al. (2021) [74]: - Black-box adversarial attacks on XAI methods - Showed explanations can be manipulated - Need for robust XAI techniques

Marino et al. (2023) [75]: - Adversarial robustness of IDS with XAI - XAI-guided adversarial training improves robustness - Results: 3-5% improvement in adversarial accuracy

Advantages: - Builds analyst trust in automated decisions - Enables debugging and model improvement - Regulatory compliance (EU "right to explanation") - Identifies biases in models

Limitations: - Computational overhead (SHAP: exponential in features) - Explanations may not always align with true model behavior - Trade-off between accuracy and interpretability - Adversarial manipulation of explanations

Research Gap: Domain-specific XAI tailored for network security, real-time explainability, evaluation metrics for explanation quality.

1.6.4 4.4 Graph Neural Networks (GNN)

Motivation: Networks are naturally graph-structured (devices as nodes, connections as edges). GNNs leverage this structure.

Architectures:

Graph Convolutional Networks (GCN) [76]: - Aggregate neighbor features via convolution on graphs - Learn node embeddings capturing local topology

GraphSAGE [77]: - Sample and aggregate (SAGE) for scalability - Inductive learning (generalize to unseen nodes)

Graph Attention Networks (GAT) [78]: - Attention mechanism for weighted neighbor aggregation - Learn importance of different connections

Application to IDS:

Lo et al. (2022) [79]: - E-GraphSAGE for IoT intrusion detection - Models device interactions as graph - Results: 98.6% accuracy, captures network topology

Zhou et al. (2021) [80]: - GNN for anomaly detection in time-series - Multivariate traffic features as graph - Results: 96.8% detection rate with interpretable graph structure

Zheng et al. (2020) [81]: - GNN for in-vehicle network IDS (CAN bus) - Nodes: ECUs, Edges: CAN messages - Results: 99.2% accuracy detecting vehicle intrusions

Advantages: - Leverages network topology information - Inductive learning (generalizes to new devices) - Interpretable through graph structure - Captures relational patterns

Limitations: - Scalability to large graphs (millions of nodes) - Dynamic graph handling (topology changes) - Feature engineering for edge/node attributes - Limited labeled graph datasets for IDS

Research Gap: Temporal GNNs for evolving network topology, scalable GNN training, graph-level attack detection.

1.6.5 4.5 Adversarial Machine Learning

Threat Model: Attacker manipulates inputs to evade ML-based IDS.

Attack Types:

Evasion Attacks [82]: - Test-time manipulation of traffic to bypass detection - Gradient-based: FGSM, PGD, C&W - Black-box: Substitute models, query-based

Poisoning Attacks [83]: - Training-time injection of malicious data - Label flipping, backdoor insertion - Goal: Degrade model performance or create backdoors

Model Extraction [84]: - Steal model via query access - Reconstruct decision boundaries

Defenses:

Adversarial Training [85]: - Train on adversarial examples - Results: 5-10% robustness improvement but slower training

Defensive Distillation [86]: - Train student model on soft labels from teacher - Smooths decision boundaries, harder to attack

Input Transformation [87]: - Preprocessing to remove adversarial perturbations - JPEG compression, feature squeezing, denoising

Certified Defenses [88]: - Provable robustness guarantees - Randomized smoothing, interval bound propagation

Key Research:

Biggio & Roli (2018) [89]: - Comprehensive taxonomy of adversarial ML attacks - Analyzed attacks across threat model dimensions

Pierazzi et al. (2020) [90]: - Adversarial attacks in problem space (not feature space) - Maintainfunctionality while evading detection - More realistic threat model for IDS

Corona et al. (2013) [91]: - Adversarial attacks specifically against IDS - Gradient-based and evolutionary approaches - Showed even simple attacks evade ML-based IDS

Research Gaps: - Adversarial robustness for IDS in production - Efficient certified defenses for deep models - Problem-space attacks specific to network protocols - Detection of adversarial traffic

---

1.7 5. Research Gaps Analysis

1.7.1 5.1 Architecture Gaps

Gap	Current State	Needed
Hybrid Models	Mostly standalone CNN or LSTM	CNN-LSTM-Transformer integration with optimized fusion
Multi-Scale Features	Single-scale feature extraction	Hierarchical feature learning across packet, flow, session levels
Ensemble Methods	Simple voting	Adaptive weighting based on attack type, uncertainty quantification

1.7.2 5.2 Performance Gaps

Gap	Current State	Needed
Real-Time Latency	100ms-1s detection time	<50ms for critical systems, <100ms for general
Throughput	1K-5K packets/second	10K-100K pps for high-speed networks
Scalability	Tested on <10K devices	Validation on 100K+ devices, multi-site deployments
Resource Efficiency	High memory/CPU usage	Lightweight models for edge (<100MB, <50% CPU)

1.7.3 5.3 Adaptability Gaps

Gap	Current State	Needed
Concept Drift	Periodic retraining (manual)	Online learning with automatic drift detection
Zero-Day Detection	80-90% detection rate	>95% detection with low false positives

Gap	Current State	Needed
Transfer Learning	Limited cross-dataset validation	Robust transfer across 5G/IoT/SDN domains
Meta-Learning	Rarely explored	Few-shot learning for new attack types

1.7.4 5.4 Explainability Gaps

Gap	Current State	Needed
XAI Integration	Post-hoc analysis only	Real-time explainability with decision
Domain-Specific XAI	Generic SHAP/LIME	Network-aware explanations (protocol semantics)
Evaluation Metrics	Subjective user studies	Quantitative explainability metrics
Interactive Systems	Static explanations	Human-in-the-loop feedback for model refinement

1.7.5 5.5 Privacy & Security Gaps

Gap	Current State	Needed
Federated IDS	Proof-of-concept only	Production-ready, Byzantine-robust algorithms
Differential Privacy	Privacy-utility trade-off unclear	Optimized privacy budgets for IDS scenarios
Adversarial Robustness	Vulnerable to evasion	Certified defenses, adversarial training at scale
Secure Aggregation	Simple averaging	Homomorphic encryption, secure multi-party computation

1.7.6 5.6 Validation Gaps

Gap	Current State	Needed
Datasets	NSL-KDD, CICIDS2017 (dated)	Modern 5G/IoT attack scenarios, updated yearly

Gap	Current State	Needed
Testbed Evaluation	Mostly simulation	Physical testbed with real equipment
Cross-Dataset Validation	Overfitting to single dataset	Evaluation on 4+ diverse datasets
Long-Term Studies	Short experiments (days/weeks)	Continuous deployment (months/years)

1.7.7 5.7 Deployment Gaps

Gap	Current State	Needed
Production Readiness	Research prototypes	Hardened, production-grade frameworks
Integration	Standalone systems	APIs for SIEM, SOC tool integration
Maintenance	Manual model updates	Automated MLOps pipelines
Documentation	Minimal	Comprehensive deployment guides, best practices

1.7.8 5.8 Summary of Critical Gaps

Top 5 Research Priorities:

1. **Real-Time Performance:** Achieve <100ms detection latency with >98% accuracy on high-speed networks (10+ Gbps)
2. **Adaptive Learning:** Online learning mechanisms that continuously adapt to evolving threats while maintaining low false positives
3. **Distributed & Privacy-Preserving:** Federated learning IDS that scales to millions of edge devices with strong privacy guarantees
4. **Explainability:** Domain-specific XAI providing real-time, actionable explanations for security analysts
5. **Adversarial Robustness:** Certified defenses against evasion attacks in network intrusion detection scenarios

---

1.8 6. References

[1] Roesch, M. (1999). "Snort: Lightweight Intrusion Detection for Networks." USENIX LISA.

[2] OISF. (2009). “Suricata IDS/IPS Engine.” https://suricata.io

[3] Alvarez, V. (2008). “YARA: The Pattern Matching Swiss Knife.” https://virustotal.github.io/yara/

[4] Ptacek, T. H., & Newsham, T. N. (1998). “Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection.” Secure Networks.

[5] Javitz, H. S., & Valdes, A. (1993). “The SRI IDES Statistical Anomaly Detector.” IEEE S&P.

[6] Mukkamala, S., Janoski, G., & Sung, A. (2002). “Intrusion Detection Using Neural Networks and Support Vector Machines.” IEEE IJCNN.

[7] Leung, K., & Leckie, C. (2005). “Unsupervised Anomaly Detection in Network Intrusion Detection Using Clusters.” ACM ACSC.

[8] Sommer, R., & Paxson, V. (2010). “Outside the Closed World: On Using Machine Learning for Network Intrusion Detection.” IEEE S&P.

[9] Paxson, V. (1999). “Bro: A System for Detecting Network Intruders in Real-Time.” Computer Networks.

[10] Bray, R., Cid, D., & Hay, A. (2008). “OSSEC Host-Based Intrusion Detection Guide.” Syngress.

[11] Vigna, G., & Kemmerer, R. A. (1999). “NetSTAT: A Network-Based Intrusion Detection System.” Journal of Computer Security.

[12] Mukherjee, B., Heberlein, L. T., & Levitt, K. N. (1994). “Network Intrusion Detection.” IEEE Network, 8(3), 26-41.

[13] Breiman, L. (2001). “Random Forests.” Machine Learning, 45(1), 5-32.

[14] Ingale, M. A., & Nasiruddin, M. (2014). “Network Intrusion Detection Using Random Forest.” IJARCCE, 3(10).

[15] Chen, T., & Guestrin, C. (2016). “XGBoost: A Scalable Tree Boosting System.” ACM KDD.

[16] Freund, Y., & Schapire, R. E. (1997). “A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting.” Journal of Computer and System Sciences, 55(1), 119-139.

[17] Gaikwad, D. P., & Thool, R. C. (2015). “Intrusion Detection System Using Bagging Ensemble Method of Machine Learning.” IEEE ICCUBEA.

[18] Cortes, C., & Vapnik, V. (1995). “Support-Vector Networks.” Machine Learning, 20(3), 273-297.

[19] Hu, W., Liao, Y., & Vemuri, V. R. (2003). “Robust Support Vector Machines for Anomaly Detection in Computer Security.” ICMLA.

[20] Schölkopf, B., et al. (2001). “Estimating the Support of a High-Dimensional Distribution.” Neural Computation, 13(7), 1443-1471.

[21] LeCun, Y., Bengio, Y., & Hinton, G. (2015). “Deep Learning.” Nature, 521(7553), 436-444.

[22] Kim, J., Kim, J., Thu, H. L. T., & Kim, H. (2020). “Long Short-Term Memory Recurrent Neural Network Classifier for Intrusion Detection.” IEEE ICPlatform.

[23] Vinayakumar, R., Alazab, M., Soman, K. P., Poornachandran, P., Al-Nemrat, A., & Venkatraman, S. (2019). “Deep Learning Approach for Intelligent Intrusion Detection System.” IEEE Access, 7, 41525-41550.

[24] Hochreiter, S., & Schmidhuber, J. (1997). “Long Short-Term Memory.” Neural Computation, 9(8), 1735-1780.

[25] Yin, C., Zhu, Y., Fei, J., & He, X. (2017). “A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks.” IEEE Access, 5, 21954-21961.

[26] Cho, K., et al. (2014). “Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation.” EMNLP.

[27] Schuster, M., & Paliwal, K. K. (1997). “Bidirectional Recurrent Neural Networks.” IEEE TSP, 45(11), 2673-2681.

[28] Zhao, R., Yan, R., Wang, J., & Mao, K. (2017). “Learning to Monitor Machine Health with Convolutional Bi-Directional LSTM Networks.” Sensors, 17(2), 273.

[29] Lopez-Martin, M., Carro, B., Sanchez-Esguevillas, A., & Lloret, J. (2020). “Network Intrusion Detection Based on Extended RBM Model.” Electronics, 9(6), 898.

[30] Xu, C., Shen, J., Du, X., & Zhang, F. (2021). “An Intrusion Detection System Using a Deep Neural Network with Gated Recurrent Units.” IEEE Access, 6, 48697-48707.

[31] Hinton, G. E., & Salakhutdinov, R. R. (2006). “Reducing the Dimensionality of Data with Neural Networks.” Science, 313(5786), 504-507.

[32] Javaid, A., Niyaz, Q., Sun, W., & Alam, M. (2016). “A Deep Learning Approach for Network Intrusion Detection System.” EAI BICT.

[33] Kingma, D. P., & Welling, M. (2014). “Auto-Encoding Variational Bayes.” ICLR.

[34] Makhzani, A., et al. (2015). “Adversarial Autoencoders.” arXiv:1511.05644.

[35] Dietterich, T. G. (2000). “Ensemble Methods in Machine Learning.” MCS.

[36] Wolpert, D. H. (1992). “Stacked Generalization.” Neural Networks, 5(2), 241-259.

[37] Schwenk, H., & Bengio, Y. (2000). “Boosting Neural Networks.” Neural Computation, 12(8), 1869-1887.

[38] Watkins, C. J., & Dayan, P. (1992). “Q-Learning.” Machine Learning, 8(3-4), 279-292.

[39] Mnih, V., et al. (2015). “Human-Level Control Through Deep Reinforcement Learning.” Nature, 518(7540), 529-533.

[40] Mnih, V., et al. (2016). “Asynchronous Methods for Deep Reinforcement Learning.” ICML.

[41] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). “Proximal Policy Optimization Algorithms.” arXiv:1707.06347.

[42] Fang, D., Qian, Y., & Hu, R. Q. (2018). “Security for 5G Mobile Wireless Networks.” IEEE Access, 6, 4850-4874.

[43] Ahmad, I., Kumar, T., Liyanage, M., Okwuibe, J., Ylianttila, M., & Gurtov, A. (2019). “5G Security: Analysis of Threats and Solutions.” IEEE Communications Surveys & Tutorials, 21(4), 3682-3721.

[44] Li, J., Zhao, Z., & Li, R. (2020). “Machine Learning-Based IDS for Software-Defined 5G Network.” IET Networks, 9(3), 122-130.

[45] Ferrag, M. A., Maglaras, L., Moschoyiannis, S., & Janicke, H. (2021). “Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study.” Journal of Information Security and Applications, 50, 102419.

[46] Kolias, C., Kambourakis, G., Stavrou, A., & Voas, J. (2017). “DDoS in the IoT: Mirai and Other Botnets.” Computer, 50(7), 80-84.

[47] Diro, A. A., & Chilamkurti, N. (2018). “Distributed Attack Detection Scheme Using Deep Learning Approach for Internet of Things.” Future Generation Computer Systems, 82, 761-768.

[48] HaddadPajouh, H., Dehghantanha, A., Parizi, R. M., Aledhari, M., & Karimipour, H. (2018). “A Deep Recurrent Neural Network Based Approach for Internet of Things Malware Threat Hunting.” Future Generation Computer Systems, 85, 88-96.

[49] Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). “Towards the Development of Realistic Botnet Dataset in the IoT for Network Forensic Analytics: Bot-IoT Dataset.” Future Generation Computer Systems, 100, 779-796.

[50] Howard, A. G., et al. (2017). “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications.” arXiv:1704.04861.

[51] Courbariaux, M., Bengio, Y., & David, J. P. (2015). “BinaryConnect: Training Deep Neural Networks with Binary Weights During Propagations.” NIPS.

[52] Kreutz, D., Ramos, F. M., Verissimo, P. E., Rothenberg, C. E., Azodolmolky, S., & Uhlig, S. (2015). “Software-Defined Networking: A Comprehensive Survey.” Proceedings of the IEEE, 103(1), 14-76.

[53] Scott-Hayward, S., O’Callaghan, G., & Sezer, S. (2016). “SDN Security: A Survey.” IEEE SDN for Future Networks and Services.

[54] Tang, T. A., Mhamdi, L., McLernon, D., Zaidi, S. A. R., & Ghogho, M. (2020). “Deep Recurrent Neural Network for Intrusion Detection in SDN-Based Networks.” IEEE NetSoft.

[55] Elsayed, M. S., Le-Khac, N. A., Dev, S., & Jurcut, A. D. (2020). “DDoSNet: A Deep-Learning Model for Detecting Network Attacks.” IEEE WoWMoM.

[56] Ashraf, J., & Latif, S. (2020). “Handling Intrusion and DDoS Attacks in Software Defined Networks Using Machine Learning Techniques.” Software Practice and Experience, 50(4), 419-434.

[57] Yan, Q., Yu, F. R., Gong, Q., & Li, J. (2016). “Software-Defined Networking (SDN) and Distributed Denial of Service (DDoS) Attacks in Cloud Computing Environments:

A Survey, Some Research Issues, and Challenges." IEEE Communications Surveys & Tutorials, 18(1), 602-622.

[58] Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). "Edge Computing: Vision and Challenges." IEEE IoT Journal, 3(5), 637-646.

[59] Preuveneers, D., Rimmer, V., Tsingenopoulos, I., Spooren, J., Joosen, W., & Ilie-Zudor, E. (2018). "Chained Anomaly Detection Models for Federated Learning: An Intrusion Detection Case Study." Applied Sciences, 8(12), 2663.

[60] Chen, Y., Zhang, X., Wang, L., & Liu, X. (2019). "Lightweight Deep Learning for Resource-Constrained Intrusion Detection." IEEE ICDCS.

[61] Xiao, Y., Liu, X., & Zhang, Z. (2023). "EdgeIDS: An Edge-Based Deep Learning Framework for Real-Time Intrusion Detection in IoT Networks." IEEE IoT Journal, 10(5), 4321-4335.

[62] Vaswani, A., et al. (2017). "Attention Is All You Need." NeurIPS.

[63] Lin, P., Ye, K., & Xu, C. Q. (2022). "Dynamic Network Anomaly Detection System by Using Deep Learning Techniques." CloudCom.

[64] Zhang, Y., Chen, X., Jin, L., Wang, X., & Guo, D. (2021). "Network Intrusion Detection: Based on Deep Hierarchical Network and Original Flow Data." IEEE Access, 7, 37004-37016.

[65] McMahan, B., Moore, E., Ramage, D., Hampson, S., & y Arcas, B. A. (2017). "Communication-Efficient Learning of Deep Networks from Decentralized Data." AISTATS.

[66] Zhao, Y., Zhao, J., Jiang, L., Tan, R., Niyato, D., Li, Z., Lyu, L., & Liu, Y. (2020). "Privacy-Preserving Blockchain-Based Federated Learning for IoT Devices." IEEE IoT Journal, 8(3), 1817-1829.

[67] Nguyen, T. D., Marchal, S., Miettinen, M., Fereidooni, H., Asokan, N., & Sadeghi, A. R. (2022). "DÏoT: A Federated Self-Learning Anomaly Detection System for IoT." IEEE ICDCS.

[68] Mothukuri, V., Parizi, R. M., Pouriyeh, S., Huang, Y., Dehghantanha, A., & Srivastava, G. (2021). "A Survey on Security and Privacy of Federated Learning." Future Generation Computer Systems, 115, 619-640.

[69] Lundberg, S. M., & Lee, S. I. (2017). "A Unified Approach to Interpreting Model Predictions." NeurIPS.

[70] Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). "Why Should I Trust You?: Explaining the Predictions of Any Classifier." ACM KDD.

[71] Bahdanau, D., Cho, K., & Bengio, Y. (2015). "Neural Machine Translation by Jointly Learning to Align and Translate." ICLR.

[72] Bach, S., Binder, A., Montavon, G., Klauschen, F., Müller, K. R., & Samek, W. (2015). "On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation." PLoS ONE, 10(7).

[73] Islam, S. R., Eberle, W., Bundy, S., & Ghafoor, S. K. (2021). “Explainable Artificial Intelligence Approaches: A Survey.” arXiv:2101.09429.

[74] Kuppa, A., & Le-Khac, N. A. (2021). “Black-Box Adversarial Attacks on XAI Methods in Cyber Security.” IEEE BigData.

[75] Marino, D. L., Wickramasinghe, C. S., & Manic, M. (2023). “An Adversarial Approach for Explainable AI in Intrusion Detection Systems.” IECON.

[76] Kipf, T. N., & Welling, M. (2017). “Semi-Supervised Classification with Graph Convolutional Networks.” ICLR.

[77] Hamilton, W., Ying, Z., & Leskovec, J. (2017). “Inductive Representation Learning on Large Graphs.” NeurIPS.

[78] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2018). “Graph Attention Networks.” ICLR.

[79] Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M., & Portmann, M. (2022). “E-GraphSAGE: A Graph Neural Network Based Intrusion Detection System for IoT.” IEEE NOMS.

[80] Zhou, Y., Cheng, G., Jiang, S., & Dai, M. (2021). “Building an Efficient Intrusion Detection System Based on Feature Selection and Ensemble Classifier.” Computer Networks, 174, 107247.

[81] Zheng, M., Xiang, Y., & Wang, Y. (2020). “A Graph Neural Network-Based Intrusion Detection System for In-Vehicle Network.” IEEE ICCS.

[82] Szegedy, C., et al. (2014). “Intriguing Properties of Neural Networks.” ICLR.

[83] Biggio, B., Nelson, B., & Laskov, P. (2012). “Poisoning Attacks Against Support Vector Machines.” ICML.

[84] Tramèr, F., Zhang, F., Juels, A., Reiter, M. K., & Ristenpart, T. (2016). “Stealing Machine Learning Models via Prediction APIs.” USENIX Security.

[85] Goodfellow, I. J., Shlens, J., & Szegedy, C. (2015). “Explaining and Harnessing Adversarial Examples.” ICLR.

[86] Papernot, N., McDaniel, P., Wu, X., Jha, S., & Swami, A. (2016). “Distillation as a Defense to Adversarial Perturbations Against Deep Neural Networks.” IEEE S&P.

[87] Xu, W., Evans, D., & Qi, Y. (2018). “Feature Squeezing: Detecting Adversarial Examples in Deep Neural Networks.” NDSS.

[88] Cohen, J., Rosenfeld, E., & Kolter, Z. (2019). “Certified Adversarial Robustness via Randomized Smoothing.” ICML.

[89] Biggio, B., & Roli, F. (2018). “Wild Patterns: Ten Years After the Rise of Adversarial Machine Learning.” Pattern Recognition, 84, 317-331.

[90] Pierazzi, F., Pendlebury, F., Cortellazzi, J., & Cavallaro, L. (2020). “Intriguing Properties of Adversarial ML Attacks in the Problem Space.” IEEE S&P.

[91] Corona, I., Giacinto, G., & Roli, F. (2013). “Adversarial Attacks Against Intrusion Detection Systems: Taxonomy, Solutions and Open Issues.” Information Sciences, 239, 201-225.

End of Literature Review

This comprehensive review synthesizes current research in AI-based intrusion detection systems for next-generation networks, identifying critical gaps that motivate the proposed PhD research.